

5a) Installation of KVM and creation of Virtual instances

Aim:

To install a KVM and creation of virtual instance for setting up a virtualized environment on linux system.

What is KVM?

KVM (Kernel-based Virtual Machine) is an open-source virtualization technology built into the Linux kernel. It turns Linux into a type-1 hypervisor that allows you to run multiple virtual machines (VMs) on a physical host.

Each VM has its own virtual hardware (CPU, RAM, disk, etc.), and runs its own full-fledged operating system.

Benefits of KVM

Feature	Benefit
Open Source	Free and supported by the Linux community.
Performance	Near-native performance using hardware virtualization (Intel VT or AMD-V).
Security	Isolation between VMs using SELinux and AppArmor.
Scalability	Can handle many VMs on a single host (scales well vertically).
Flexibility	Supports many OSes (Linux, Windows, BSD, etc.).
Integration	Works with tools like libvirt, virt-manager, OpenStack, etc.
Live Migration	Supports live VM migration between hosts.

Procedure:

Virtualization Technology

1. Check Hardware Virtualization Support.

```
admin1@admin1-VirtualBox:~/praveen111$ egrep -c 'lm' /proc/cpuinfo
fo
1
```

Output ≥ 1 indicates support. If 0, CPU doesn't support hardware virtualization.

2. Run `uname -a`, which shows kernel name, hostname, kernel version, build date, architecture, and OS type in one line.

```
admin1@admin1-VirtualBox:~/praveen111$ uname -a
Linux admin1-VirtualBox 6.5.0-27-generic #28~22.04.1-Ubuntu SMP
PREEMPT_DYNAMIC Fri Mar 15 10:51:06 UTC 2 x86_64 x86_64 x86_64 G
NU/Linux
```

3. To list the KVM kernel module files present in your kernel, run:

```
admin1@admin1-VirtualBox:~$ find /lib/modules/$(uname -r) -name
"kvm*.ko*"
/lib/modules/6.5.0-27-generic/kernel/arch/x86/kvm/kvm.ko
/lib/modules/6.5.0-27-generic/kernel/arch/x86/kvm/kvm-intel.ko
/lib/modules/6.5.0-27-generic/kernel/arch/x86/kvm/kvm-amd.ko
/lib/modules/6.5.0-27-generic/kernel/drivers/gpu/drm/i915/kvmgt.
ko
```

KVM Installation

1. Install required packages : `qemu-kvm`, `libvirt-bin`, `bridge-utils`, `virt-manager` and `qemu-system`.

```
admin1@admin1-VirtualBox:~/praveen111$ sudo apt update
sudo apt install qemu-kvm libvirt-daemon-system libvirt-clients
bridge-utils virt-manager
Hit:1 http://in.archive.ubuntu.com/ubuntu jammy InRelease
Hit:2 http://security.ubuntu.com/ubuntu jammy-security InRelease
Hit:3 http://in.archive.ubuntu.com/ubuntu jammy-updates InReleas
e
Hit:4 http://in.archive.ubuntu.com/ubuntu jammy-backports InRele
ase
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
558 packages can be upgraded. Run 'apt list --upgradable' to see
them.
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Note, selecting 'qemu-system-x86' instead of 'qemu-kvm'
bridge-utils is already the newest version (1.7-1ubuntu3).
virt-manager is already the newest version (1:4.0.0-1).
libvirt-clients is already the newest version (8.0.0-1ubuntu7.18
```

2. Configure the libvirt daemon configuration file for remote access & socket permissions in KVM/libvirt

```
admin1@admin1-VirtualBox:~/praveen111$ sudo nano /etc/libvirt/li
bvirtd.conf
```

```
keshoo@tce-VirtualBox:~$ sudo nano /etc/libvirt/libvirtd.conf
[sudo] password for keshoo:
keshoo@tce-VirtualBox:~$ grep ^[#] /etc/libvirt/libvirtd.conf
listen_addr = "0.0.0.0"
unix_sock_group = "libvirt"
unix_sock_ro_perms = "0777"
unix_sock_rw_perms = "0777"
unix_sock_dir = "/var/run/libvirt"
auth_unix_ro = "none"
auth_unix_rw = "none"
keshoo@tce-VirtualBox:~$
```

After configuring it, restart libvirtd service.

```
admin1@admin1-VirtualBox:~/praveen111$ sudo systemctl restart libvirtd
```

3. Run virsh list - Displays no of Virtual machine running.

```
admin1@admin1-VirtualBox:~/praveen111$ virsh list
Id      Name      State
-----
```

4. Connecting to Virtualization interactive terminal.

```
admin1@admin1-VirtualBox:~/praveen111$ virsh
Welcome to virsh, the virtualization interactive terminal.

Type:  'help' for help with commands
       'quit' to quit

virsh #
```

5. Getting Version of libvirt & QEMU.

```
virsh # version
Compiled against library: libvirt 8.0.0
Using library: libvirt 8.0.0
Using API: QEMU 8.0.0
Running hypervisor: QEMU 6.2.0
```

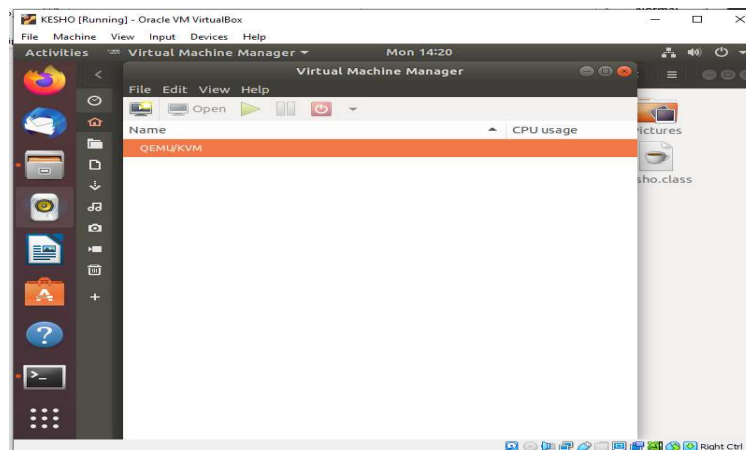
6. Getting Node information

```
virsh # nodeinfo
CPU model:      x86_64
CPU(s):         1
CPU frequency:  2207 MHz
CPU socket(s):  1
Core(s) per socket: 1
Thread(s) per core: 1
NUMA cell(s):   1
Memory size:    4001272 KiB

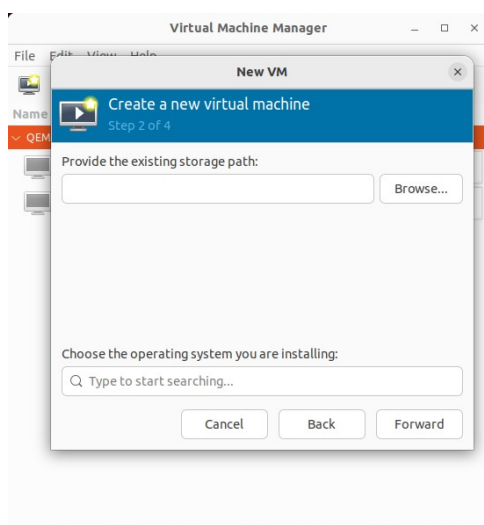
virsh #
```

Importing existing VM

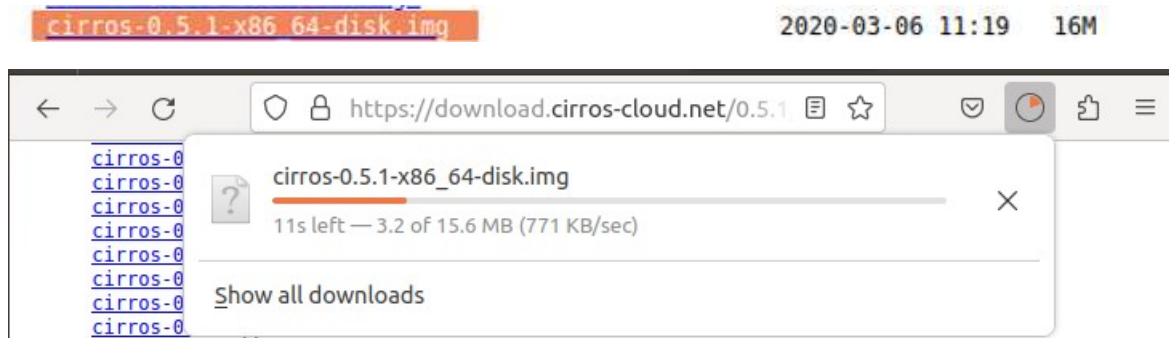
1. Go to virtual machine manager (virt-manager) in your linux system. Then, right click QEMU/KVM and select New to create new VM.



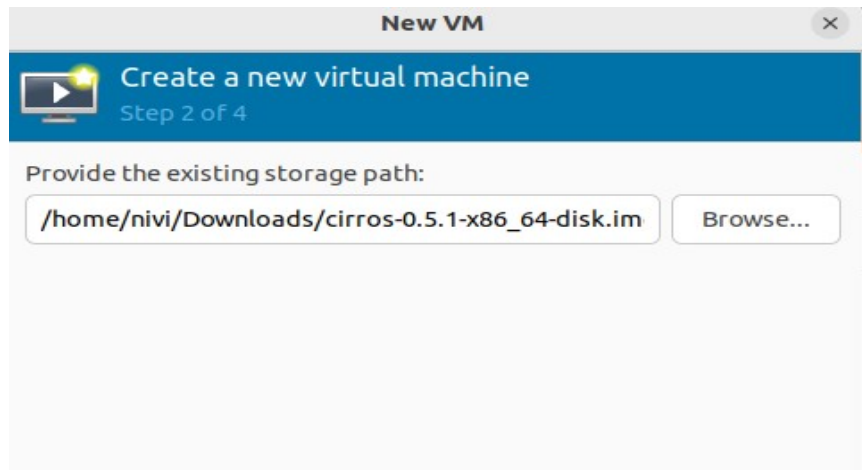
2. Select import existing disk image and click forward.



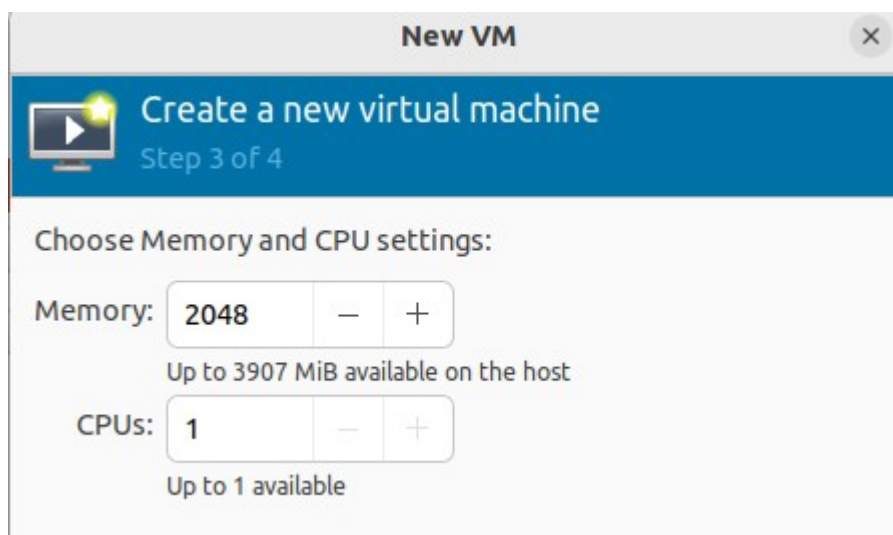
3. Download cirros disk image from the link: <https://download.cirros-cloud.net/0.5.1/>



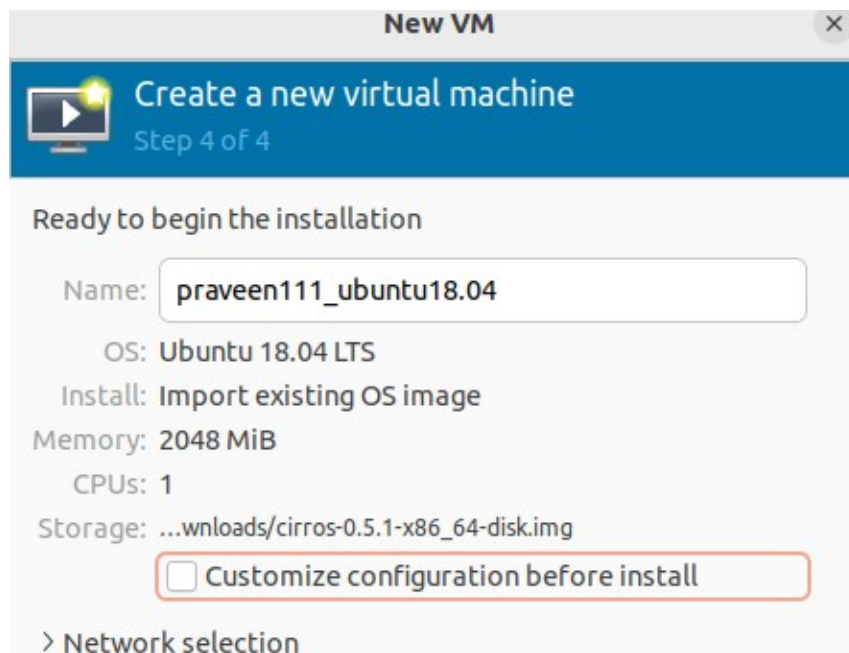
4. Give the path of downloaded cirros disk image and click forward.



5. Choose memory and CPU settings and click forward.



6.Name your VM and click Finish.



7.The VM is booted successfully and the login was also successful.

```
further output written to /dev/ttyS0
[  4.877260] virtio_blk virtio3: [vda] 229376 512-byte logical blocks (117
112 MiB)
[  5.531868] random: fast init done
[  5.535285] random: crng init done

login as 'cirros' user. default password: 'gocubsgo'. use 'sudo' for root.
cirros login: cirros
Password:
$
```

Result:

Thus, the KVM was successfully installed on the Linux system, and virtual instances were created and configured using Virt-Manager, demonstrating the setup of a virtualized environment.